

1) Consider the following user-level solution to the critical section problem, where **flag** and **turn** are shared variables initialized to **false** and **0**, respectively.

Process P0

```
while(1){  
    flag=true;  
    while(turn==1);  
    critical-section  
    turn=1;  
    remainder-section  
}
```

Process P1

```
while(1){  
    turn=0;  
    while(flag and turn==0);  
    critical-section  
    flag=false;  
    remainder-section  
}
```

Determine which of the three requirements in part (a) are not satisfied with justifications.

Critical Section (1)

- This solution does not guarantee mutual exclusion.
Consider the following sequence:

1. P1: turn=0;
2. P1: while(flag and turn==0);
3. P1: critical-section
- 4. Context switch from P1 to P0**
5. P0: flag=true;
6. P0: while(turn==1);
7. critical-section

→ **P0 and P1 are both in the critical section** 😞

Critical Section (1)

- This solution does not guarantee progress. Consider the following sequence:

1. **P0: executes the while loop once and turn becomes 1**
2. P0: flag=true;
3. P0: while(turn==1); // loop forever

→ **Progress is not satisfied for process P0** 😞

Critical Section (1)

- This solution satisfies bounded waiting.

1. P0: waiting @ while(turn==1);

In this case, P0 will enter the moment P1 executes the first statement inside its while loop. So P1 can enter its critical section 0 times after P0 has requested access.

2. P1: waiting @ while(flag and turn==0);

In this case, P1 will enter the moment P0 executes turn=1 statement after its critical section. So P0 can enter its critical section 1 time after P1 has requested access.

2. Indicate whether the following statements are true or false. Justify your answers.
 - a) Mutual exclusion can be achieved by disabling interrupts during the critical section.
 - b) If a solution to a critical section problem satisfies progress, then it also satisfies bounded waiting.

a) Mutual exclusion can be achieved by disabling interrupts during the critical section.

→ True.

Justification: By disabling interrupts, no other process, including the operating system, will be able to run during the critical section.

b) If a solution to a critical section problem satisfies progress, then it also satisfies bounded waiting.

→ False.

Justification: A solution that always favors a process P0 over another process P1 can satisfy progress without satisfying bounded waiting. In this case, the bounded waiting requirement for process P1 will be violated. Consider the following, with **flag** initialized to **false** and **timeout** some large value.

Process P0

```
while(flag);
```

```
flag=true;
```

```
critical section;
```

```
flag=false;
```

Process P1

```
while(flag) {wait(timeout);}
```

```
flag=true;
```

```
critical section;
```

```
flag=false;
```

TestAndSet

3) Consider the following implementation of **TestAndSet()**.

```
boolean TestAndSet (boolean *target) {  
    boolean rv = *target;  
    *target = false;  
    return rv ;  
}
```

Consider the following hardware-level solution to the critical section problem, where the code for a process **P_i** (**i = 0 or 1**) is shown. A **lock** is a Boolean variable. The initial value of **lock=True**.

Process P_i:

```
While(1){  
    while(TestAndSet(&lock)==false);  
    critical-section  
    lock = false;  
    remainder-section  
}
```

Does this solution satisfy the properties of mutual exclusion and progress? Justify your answers.

TestAndSet (3)

Mutual Exclusion: It satisfies mutual exclusion. If either P0 or P1 enters the critical section, the other process cannot enter critical section (lock=false). While(1){

```
    while(TestAndSet(&lock)==false);  
    critical-section  
    lock = false;  
    remainder-section  
}
```

Progress: It doesn't satisfy progress. After either P0 or P1 executes its critical section as the lock gets the value of false (the critical section remains locked), the other process is indefinitely waiting to enter.

```
While(1){  
    while(TestAndSet(&lock)==false);  
    critical-section  
    lock = false;  
    remainder-section  
}
```